

[블록체인 기반 티켓 재판매 시스템]

소프트웨어설계서

- 곽도혁 -

<https://github.com/sunnyside-up0102/blockchain-ticketing-system/tree/main>

문서 번호 : 소프트웨어공학-2026-005

소속 : 한국항공대학교

작성자 : 곽도혁

제/개정 이력

버전	날짜	작성자 성명	제/개정사항	비 고
01	20260311	곽도혁	프로젝트 정의서 작성	HW1
02	20260410	곽도혁	프로젝트 관리 계획서 작성	HW 2
03	20260506	곽도혁	요구사항 정의서	HW 3
04	20260508	곽도혁	요구사항 분석서	HW 4
05	20260524	곽도혁	소프트웨어설계서	HW 5

목 차

1. 서론 -----	1
1.1 목적 및 범위 -----	1
1.2 용어 정의 -----	1
1.3 참조 문서 -----	1
2. 소프트웨어 아키텍처 -----	2
2.1 정적 구조 -----	2
2.2 동적 구조 -----	4
3. 모듈/패키지 설계 -----	5
3.1 SDD-M/p-001 : 모듈 혹은 패키지 이름 -----	5
3.2 SDD-M/P-002 : 모듈 또는 패키지 이름 -----	5
3.3 SDD-M/P-00x : 모든 모듈/패키지에 대하여 작성 -----	5
4. 인터페이스 설계 -----	8
4.1 외부 시스템 인터페이스 -----	8
4.2 사용자 인터페이스 -----	8

5. 데이터 설계----- 9

6. 구현 기술 설계 ----- 10

7. 요구사항 추적표 ----- 11

8. 부록 ----- 12

1. 서론

1.1 목적 및 범위

본 문서는 '블록체인 기반 티켓 재판매 시스템' 프로젝트의 소프트웨어 컴포넌트에 대한 설계 사항을 기록하고 문서화 하는 데 목적이 있다. 본 설계서의 범위는 프론트엔드 웹 인터페이스, 블록체인 스마트 컨트랙트, 외부 디지털 지갑 연동 모듈, 현장 입장 유효성 검증 프로토콜 전반을 아우르며, 요구사항 정의서(Doc-003) 및 분석서(Doc-004)에서 명세화된 기능을 실제 구현 가능한 형태의 구조로 변환한다.

1.2 용어 정의

용어	설명
NFT	대체 불가능한 토큰(Non-Fungible Token)으로, 본 시스템에서는 각 티켓의 고유성 및 소유권 정보를 블록체인 상에 영구적으로 증명하기 위해 사용됨 (ERC-721 표준 준수).
스마트 컨트랙트	블록체인 네트워크 상에서 자동 실행되는 계약 코드로, 중개자 없이 티켓 민팅, 소유권 이전 및 재판매 상한가 검증을 안전하게 집행함.
가스비(Gas Fee)	블록체인 네트워크에서 상태 변경(트랜잭션)을 일으킬 때 연산 자원의 대가로 노드에 지불해야 하는 가상자산 기반 수수료.
동적 QR 코드	사용자의 실시간 소유자 지갑 주소와 타임스탬프를 암호화 조합하여 1분 주기로 자동 재생성되는 보안 입장 코드.

1.3 참조 문서

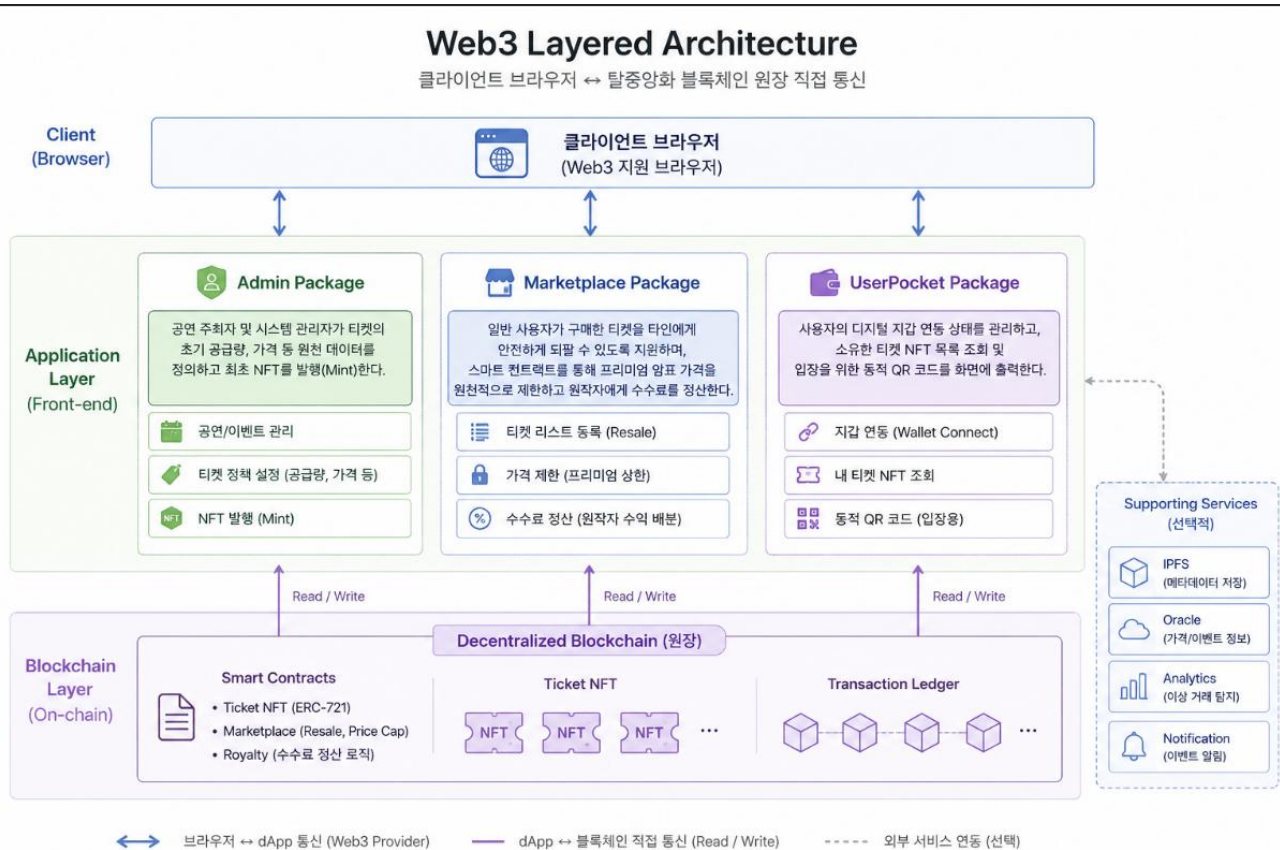
[곽도혁]시스템정의서_20260311_Doc-001.hwp

[곽도혁]프로젝트관리계획서_20260410_Doc-002.hwp

[곽도혁]요구사항정의서_20260506_Doc-003.hwp

[곽도혁]요구사항분석서_20260508_Doc-004.hwp

2. 소프트웨어 아키텍처



패키지명	기능 설명	담당자
Admin	공연 주최자 및 시스템 관리자가 티켓의 초기 공급량, 가격 등 원천 데이터를 정의하고 최초 NFT를 발행(Mint)한다.	공동
Marketplace	일반 사용자가 구매한 티켓을 타인에게 안전하게 되팔 수 있도록 지원하며, 스마트 컨트랙트를 통해 프리미엄 암표 가격을 원천적으로 제한하고 원작자에게 수수료를 정산한다.	공동
UserPocket	사용자의 디지털 지갑 연동 상태를 관리하고, 소유한 티켓 NFT 목록 조회 및 입장을 위한 동적 QR 코드를 화면에 출력한다.	공동

2.2 동적 구조

시스템의 핵심 비즈니스 로직 흐름인 [티켓 구매 및 소유권 변경]의 런타임 객체 상호작용 흐름은 다음과 같다.

1. r Interface에서 사용자가 티켓 구매를 요청하면 Digital Wallet(지갑 모듈)에 트랜잭션 서명을 요청한다.
2. 승인이 완료되면 서명된 트랜잭션이 Smart Contract(스마트 컨트랙트)로 전송된다.
3. Smart Contract는 조건 및 가스비를 검증한 후 Ticket NFT의 내부 상태 정보를 전송(소유권 이전)한다.
4. Ticket NFT의 상태가 변경되면 최종적으로 모든 내역이 Blockchain Network(원장)에 영구 기록 및 확정된다.

3. 모듈/패키지 설계

3.1 SDD-M-001 : Admin (관리자 및 발행 패키지)

3.1.1 모듈 설명

먼저 소프트웨어를 구성하는 패키지에 대하여 간단히 정의합니다. 예를 들면 다음과 같습니다.

패키지설명	공연 주최자 및 시스템 관리자가 티켓 정보를 등록하고 최초 NFT를 민팅하기 위한 패키지.	
구성 클래스	C01_Manager	관리자 권한 및 세션을 검증하고 대시보드 접근을 제어하는 클래스.
	TicketNFT	ERC-721 사양에 따라 고유 티켓 토큰을 생성하고 관리하는 클래스.

3.1.2 구성 클래스 설계

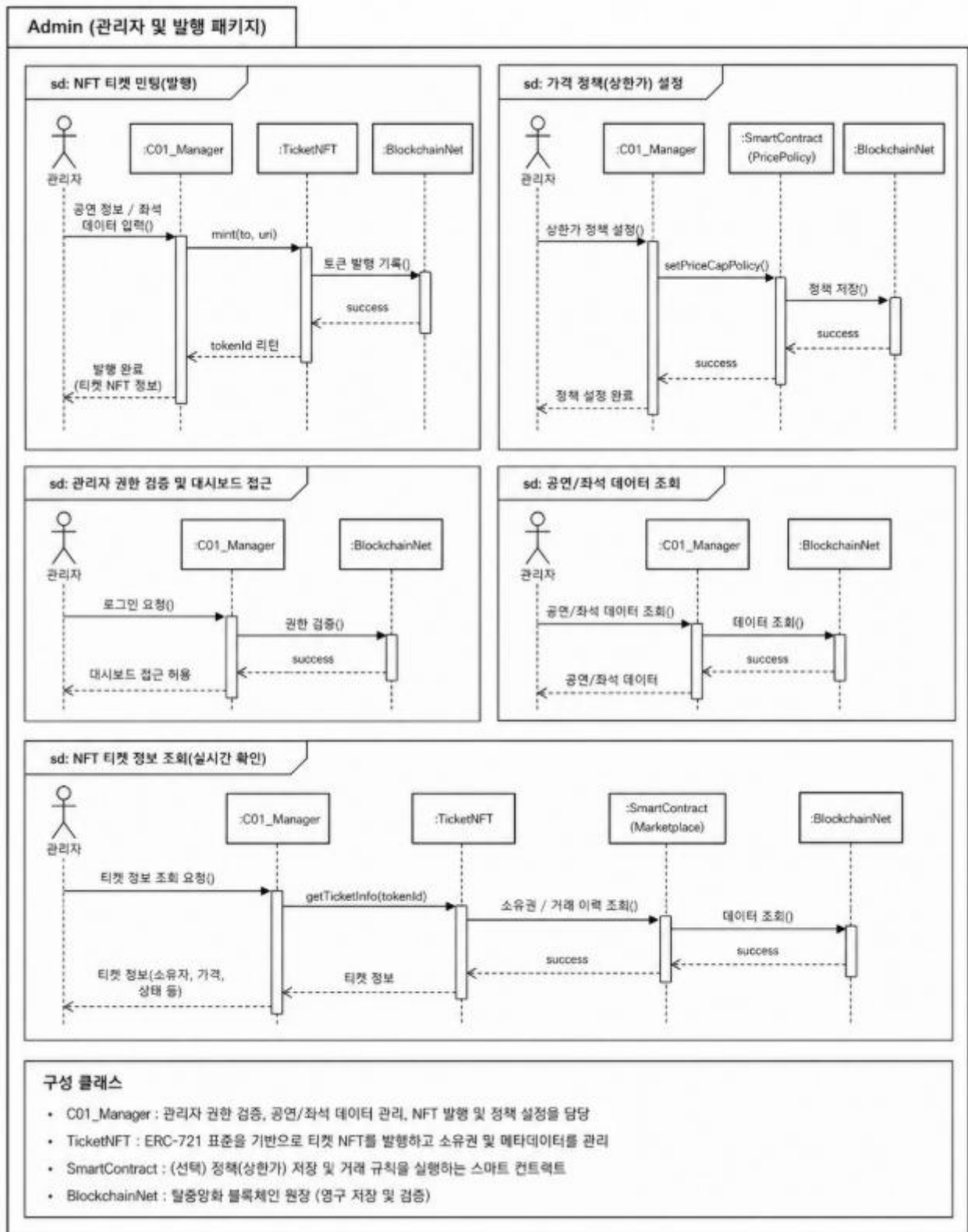
(1) CM-001 : C01_Manager

클래스 타입	Concrete	관련 Use Case	U_01, U_09
설 명	관리자가 공연 정보와 좌석 데이터를 기반으로 블록체인상에 티켓 NFT를 생성 및 검증하는 권한 클래스.		
멤버 변수	- managerID : String - institutionName : String		
멤버 함수	+ mintTicketNFT() : void + setPriceCapPolicy() : void		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : TicketNFT, BlockchainNet		

(2) CM-002: TicketNFT

클래스 타입	Concrete	관련 Use Case	: U_01, U_02, U_04, U_07
설 명	ERC-721 표준 인터페이스를 상속받아 블록체인 상에 고유한 티켓 토큰을 생성하고, 소유권 정보 및 메타데이터(공연 정보)를 영구 원장에 유지·관리하는 핵심 자산 클래스.		
멤버 변수	- tokenId : uint256 (티켓 고유 식별 번호, Primary Key) - metadataURI : String (공연명, 일시, 좌석 정보가 저장된 IPFS 주소 링크) - initialPrice : uint256 (최초 민팅 가격, Wei 단위) - currentOwner : address (현재 티켓을 보유하고 있는 사용자의 지갑 주소)		
멤버 함수	+ mint(to: address, uri: String) : uint256 (새로운 티켓 NFT 발행) + transferFrom(from: address, to: address, tokenId: uint256) : void (소유권 이전 승인 및 실행) + getInitialPrice(tokenId: uint256) : uint256 (상한가 검증을 위한 최초 발행가 리턴)		
관계성	Generalization(a-kind-of) : ERC721 Standard Contract Aggregation (has-parts) : Other Associations : C01_Manager (발행 주체), C03_SmartContract (거래 중계), BlockchainNet (원장 저장)		

3.1.3 패키지 행위



3.1.4 멤버함수(메소드) 설계

(1) M-001 : mintTicketNFT()

소속 클래스	CM-001 : C01_Manager
트리거 클래스	Web3 UI Dashboard
파라미터	공연 정보, 초기 가격, 발행 수량
세부 처리 로직	
1. 관리자가 대시보드 화면에 공연명, 일시, 좌석 정보, 초기 가격 및 발행 수량을 입력한다. 2. 관리자가 '티켓 NFT 발행' 버튼을 클릭하여 승인 이벤트를 발생시킨다. 3. 시스템은 권한이 유효한 관리자 세션인지 최종 확인한 후, 자산 생성을 위해 TicketNFT 클래스의 mint() 함수를 호출한다.	

(2) M-002 : setPriceCapPolicy()

소속 클래스	CM-001 : C01_Manager
트리거 클래스	Web3 UI Dashboard
파라미터	재판매 상한가 비율 (기본값: 110%)
세부 처리 로직	
1. 관리자는 암표 방지를 위해 최초 발행가 대비 거래 가능한 최대 상한선 비율을 설정한다. 2. 입력된 상한가 정책 데이터는 마켓플레이스 스마트 컨트랙트(C03_SmartContract)의 가격 제어 변수로 연동 및 반영되도록 시스템 파라미터를 전송한다.	

(3) M-003 : mint()

소속 클래스	CM-002 : TicketNFT
트리거 클래스	CM-001 : C01_Manager
파라미터	관리자 지갑 주소, 메타데이터 URI, 초기 가격
세부 처리 로직	
1. 관리자 클래스로부터 호출 요청을 받아 내부 카운터를 통해 순차적인 고유 토큰 ID(tokenID)를 자동 생성한다. 2. 입력받은 초기 가격 정보(initialPrice)를 해당 토큰 ID의 상태 데이터에 매핑하여 저장한다. 3. 시스템은 블록체인 네트워크(BlockchainNet)에 ERC-721 규격의 민팅 트랜잭션을 실행하고, 영구적으로 토큰을 온체인(On-chain) 원장에 기록한다.	

(4) M-004 : getInitialPrice()

소속 클래스	CM-002 : TicketNFT
트리거 클래스	CM-03_SmartContract (마켓플레이스 클래스)
파라미터	토큰 ID (tokenID)
세부 처리 로직	
1. 외부 마켓플레이스 컨트랙트에서 특정 티켓의 재판매 허용 범위를 검증하기 위해 본 함수를 호출한다. 2. 상태 데이터 원장 조회를 통해 입력받은 tokenID에 해당하는 initialPrice(최초 발행 단가) 값을 찾아 호출 측에 안전하게 리턴한다.	

3.2 SDD-P-002: Information

3.2.1 패키지 설명

패키지설명	발행된 티켓 NFT의 안전 거래 및 암호 방지 가격 제한 정책(110% 상한선)과 판매 대금 정산을 책임지는 스마트 컨트랙트 비즈니스 로직 패키지이다.	
구성 클래스	스마트 컨트랙트	마켓플레이스 판매 등록 및 가격 검증, 결제 정산을 처리하는 핵심 클래스이다.
	수수료 정산기	재판매 거래 성립 시 원작자 로열티와 판매자 대금을 자동 분할 정산하는 클래스이다.

3.2.2 구성 클래스 설계

(1) CL-001 : 스마트 컨트랙트 (Smart Contract)

클래스 타입	Concrete	관련 Use Case	U_02, U_04, U_05
설 명	티켓의 양도 계약을 자동화하고, 사용자가 입력한 재판매 가격이 초기 발행가 대비 110%를 초과하지 않는지 철저히 검증 및 제어한다		
멤버 변수	contractAddress : String priceCapPercentage : Integer		
멤버 함수	+ verifyPriceCap(tokenID, proposedPrice) : Boolean + transferOwnership(tokenID, buyerAddress) : void		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : TicketNFT Other Associations : C02_User, C05_DigitalWallet, BlockchainNet		

(2) CL-002 : 수수료 정산기

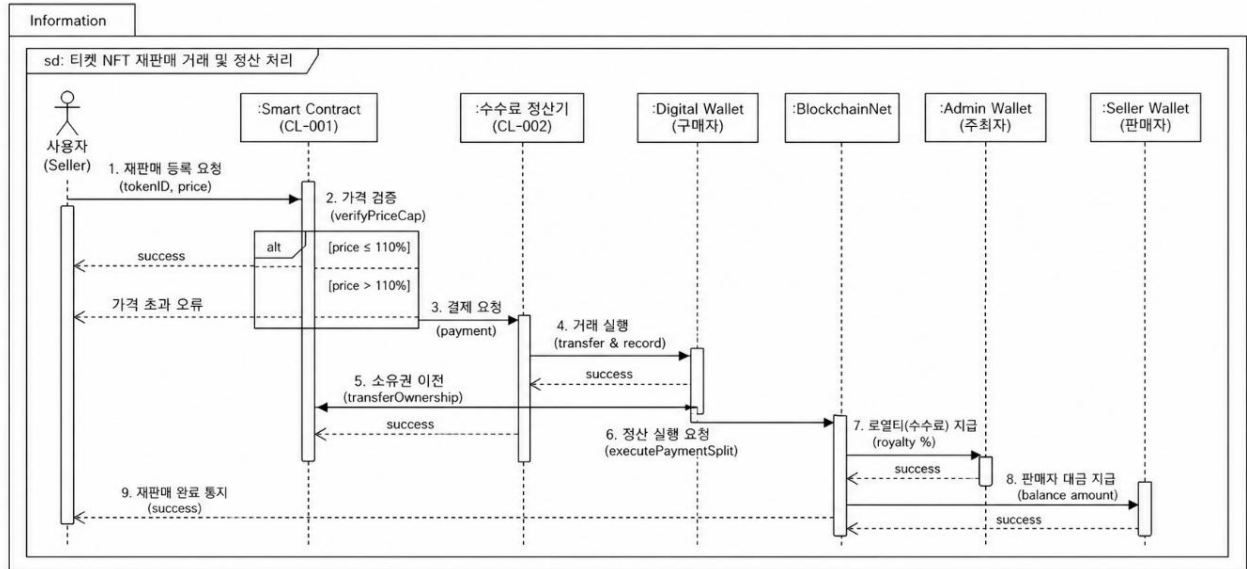
클래스 타입	Concrete	관련 Use Case	U_06
설 명	재판매 마켓플레이스에서 거래가 확정되었을 때 온체인 상에서 시스템 수수료(로열티)를 주최자 지갑으로 우선 분할 입금한 후, 나머지 자금을 판매자에게 정산한다.		

블록체인 기반 티켓 재판매 시스템]

멤버 변수	- royaltyPercentage : Integer = 5 - adminWalletAddress : String
멤버 함수	+ executePaymentSplit(tokenID) : void
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : SmartContract, BlockchainNet, Admin_Wallet, Seller_Wallet

3.2.3 패키지 행위

3.2.3 패키지 행위



3.2.4 멤버함수(메소드) 설계

(1) M-005 : executePaymentSplit()

소속 클래스	CL-002 : 수수료 정산기
트리거 클래스	CL-001 : 스마트 컨트랙트
파라미터	토큰 ID, 구매 대금
세부 처리 로직	
1. SmartContract로부터 재판매 거래 완료 승인을 받아 정산 로직을 트리거한다. 2. 입력받은 구매 대금 중 지정된 수수료 비율(royaltyPercentage = 5)만큼 계산하여 주최자 지갑 주소(adminWalletAddress)로 먼저 온체인 전송한다. 3. 수수료를 제외한 나머지 순수 대금을 판매자 지갑 주소(sellerWalletAddress)로 전송하여 온체인 분할 정산을 완료한다.	

3.3 SDD-P-003: User

3.3.1 패키지 설명

패키지설명	사용자가 자신의 디지털 지갑을 연동하여 보유한 티켓 NFT 목록을 실시간 조회하고, 안전한 현장 입장을 위한 보안 동적 QR 코드를 생성 및 관리하는 패키지이다.	
구성 클래스	디지털 지갑	외부 지갑 서비스와 연동하여 공개 지갑 주소를 확인하고 거래 서명을 중계하는 클래스이다
	티켓 검증기	공연 현장에서 사용자가 제시한 QR 코드를 실시간 스캔하여 블록체인 상의 소유권 정보를 검증하는 클래스이다.

3.3.2 구성 클래스 설계

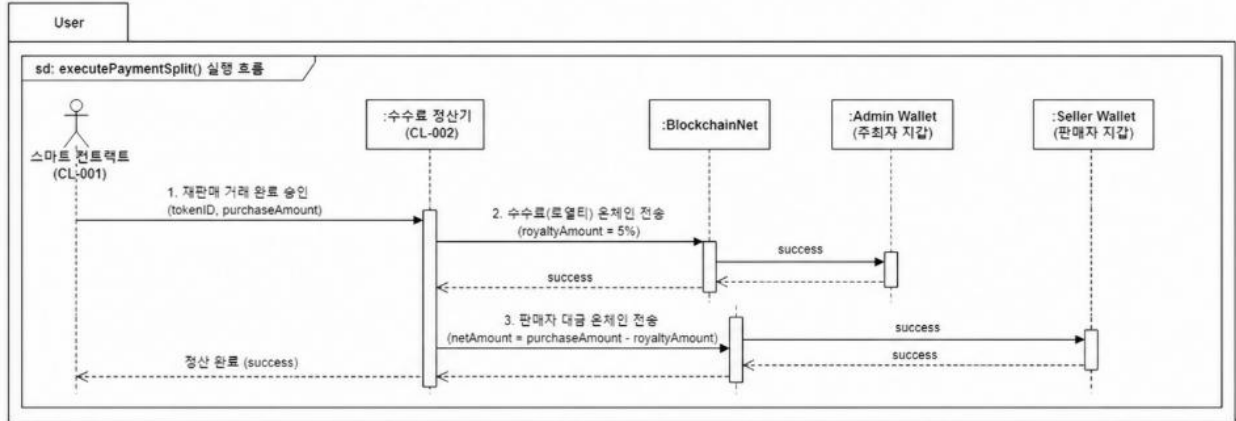
(1) CN-001: 디지털 지갑 (Digital Wallet)

클래스 타입	Concrete	관련 Use Case	U_02, U_3, U_04
설 명	사용자의 개인키 서명 환경을 제공하고, 블록체인 네트워크와 연동할 수 있도록 안전한 외부 인터페이스를 매핑한다.		
멤버 변수	- walletAddress : String - chainID : Integer		
멤버 함수	+ connectWallet() : void + signTransaction() : String		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : C02_User, SmartContract, BlockchainNet		

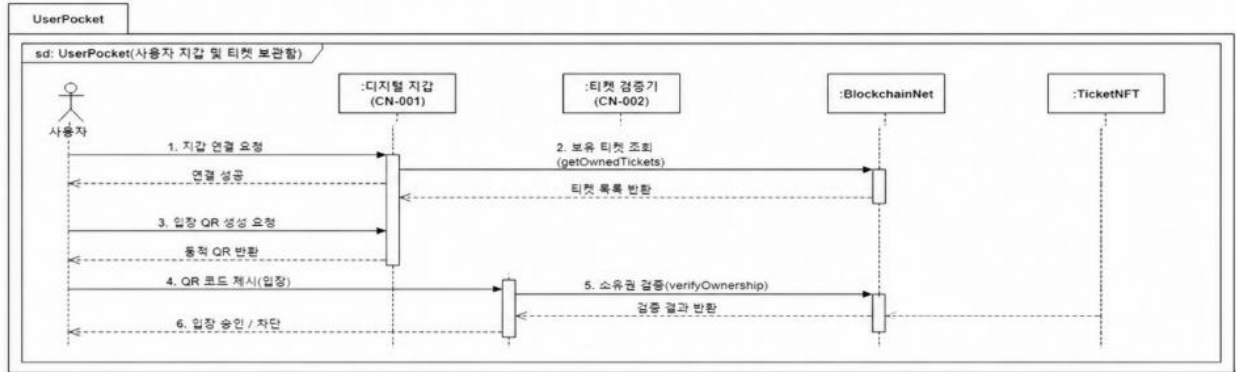
3.3.3 패키지 행위

v

▶ 3.2.4 패키지 멤버함수(메소드) 설계 (1) M-005 : executePaymentSplit()



▶ 3.3.2 구성 클래스 설계



3.3.4 멤버함수(메소드) 설계

(1) M-006 : signTransaction()

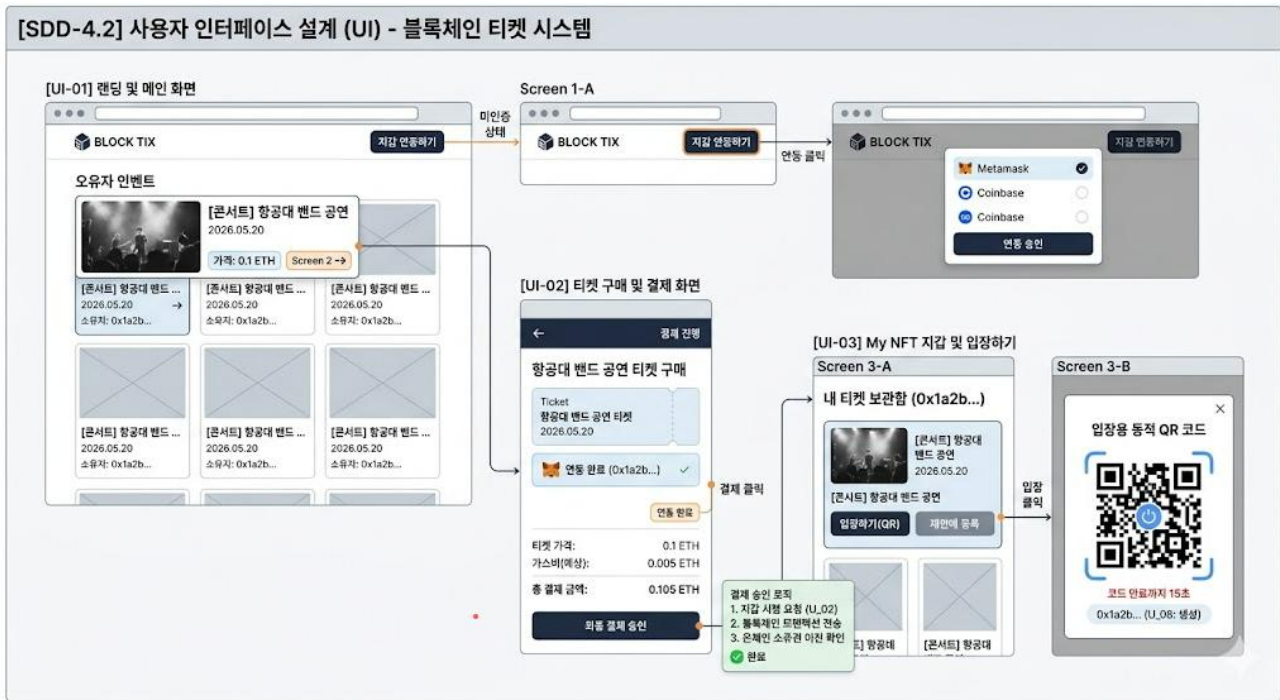
소속 클래스	CN-001 : 디지털 지갑
트리거 클래스	Web3 UI 프론트엔드
파라미터	트랜잭션 데이터 객체
세부 처리 로직	
1. 사용자가 웹 화면에서 '티켓 구매' 또는 '재판매 등록' 버튼을 클릭한다. 2. 시스템은 브라우저에 인젝션된 메타스크(Metamask) API를 호출하여 서명 팝업을 트리거한다. 3. 사용자가 지갑에서 서명을 승인하면, 개인키로 암호화된 서명값(Signature)을 웹 서버 및 스마트 컨트랙트로 반환하여 거래를 실행시킨다.	

4. 인터페이스 설계

4.1 외부 시스템 인터페이스

메시지 명	송신 모듈	수신 모듈	메시지 형식	전송방식
Wallet Sign Request	Presentation UI (웹 프론트 엔드)	Metamask Extension (디지털 지갑)	JSON-RPC 2.0	window.ethereum Injected API 브릿지
On-chain Query	Application Core (마켓플레이스 모듈)	Infura / Alchemy (블록체인 노드)	Hexadecimal String	HTTPS / WebSockets 통신 (JSON-RPC)
Transaction Event	Blockchain Network (분산 원장)	Web3 Event Listener (시스템 알림)	Solidity Event Log	JSON-RPC Pub/Sub (웹소켓 기반 실시간 스트리밍)

4.2 사용자 인터페이스



5. 데이터 설계

(1) DD-01, 티켓 NFT 정보 (On-chain 블록체인 상태 데이터)

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	Token_ID	Uint256	> 0	자동 생성	티켓 NFT 고유 식별 번호 (Primary Key)
2	Owner_Address	Address	지갑 주소	0x000...	현재 티켓

블록체인 기반 티켓 재판매 시스템

					소유자의 디지털 지 갑 주소
3	Initial_Price	Uint256	≥ 0	not Null	최초 발행 단가 (Wei 단위 저장)
4	Is_For_Sale	Boolean	true / false	false	마켓플레 이스 재판 매 등록 상태 플레 그

(2) DD-02, 티켓 메타데이터 정보 (Off-chain IPFS 저장 데이터)

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	Event_Name	String	UTF-8 문자열	not Null	공연 및 행사 명칭
2	Show_Date	Uint256	타임스탬프	not Null	공연 시작 일시 (UTC 기준 영구 기록)
3	Seat_Info	String	좌석 번호	not Null	객석 구역 및 고유 좌석 정보 (예: R석 A-12)
4	Token_ID	Uint256	> 0	not Null	상태 데이 터 매핑용 고유 식별 자

(3) DD-03, 사용자 및 관리자 계정 정보 (시스템 연동 데이터)

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	User_Address	Address	지갑 주소	not Null	사용자의 공개 디지털 지갑 주소 (Unique)
2	Email	String	이메일 형식	Null	알림 수신 용 이메일 주소 (선택 입력)
3	Account_Type	Enum	Admin, User	User	시스템 접근 권한 등급 구분 식별자

6. 구현 기술 설계

구분	클라이언트 (Frontend)	서버 및 노드 (Blockchain)	비고
구현 언어	- JavaScript - HTML5 / CSS3	- Solidity (v0.8.20)	스마트 컨트랙트 최적화 및 웹 인터페이스 표준 준수
운영 체제	- Windows - macOS / Linux - Android / iOS	- Linux (Ubuntu Server) - EVM (Ethereum Virtual Machine)	클라이언트 단말 독립성 보장 및 분산 가상 머신 가동
특수 소프트웨어	- React.js (v18)	- Hardhat (테스트 프레임워크)	탈중앙화 라이브러리

블록체인 기반 티켓 재판매 시스템]

	- Ethers.js (v6)	크) - Go-Ethereum (Geth)	및 온체인 네트워크 통합
하드웨어	- Intel i5 이상 스마트 PC - 스마트폰 모바일 단말기	- AWS EC2 클라우드 인스턴스 - Infura / Alchemy 엔드포인트	고성능 분산 데이터 노드 및 사용자 접근 기기 이원화
네트워크	- HTTPS / WebSockets	- Ethereum Sepolia Testnet	P2P 분산 네트워크 가스비 최적화 프로 토콜 프로비저닝

7. 요구사항 추적표

모듈/클래스명요구사항식별자	M-001	M-002	M-003	M-004	M-005	M-006
FR-001 (이메일 계정 생성)						
FR-002 (디지털 지갑 연동)						●
FR-003 (관리자 로그인)						
FR-004 (관리자 티켓 등록)	●					
FR-005 (티켓 NFT 발행)			●			
FR-006 (메타데이터 수정)						
FR-007 (판매 목록 결제)						●
FR-008 (소유권 이전 실행)					●	
FR-009 (상한가 범위 설정)		●				
FR-010 (상한가 초과 거부)				●		
FR-011 (거래/소유 이력 조회)						
FR-012 (일회용 보안 QR 생성)						
FR-013 (실시간 QR 소유권 대조)				●		

가로축 상단 (설계 메소드 식별자):

- M-001 : mintTicketNFT() (티켓 NFT 발행 요청)
- M-002 : setPriceCapPolicy() (재판매 상한가 정책 설정)
- M-003 : mint() (ERC-721 온체인 토큰 생성)
- M-004 : getInitialPrice() (최초 발행가 역조회)
- M-005 : executePaymentSplit() (수수료 및 대금 분할 정산)
- M-006 : signTransaction() (디지털 지갑 트랜잭션 개인키 서명)

8. 부록

- 본 설계도서는 UML 2.0 및 이더리움 ERC-721 비동기 트랜잭션 표준 규격을 준수하여 작성되었으며, 상세 인터페이스 정의는 추후 구현 단계의 API 스펙과 100% 호환됨을 보증함.